

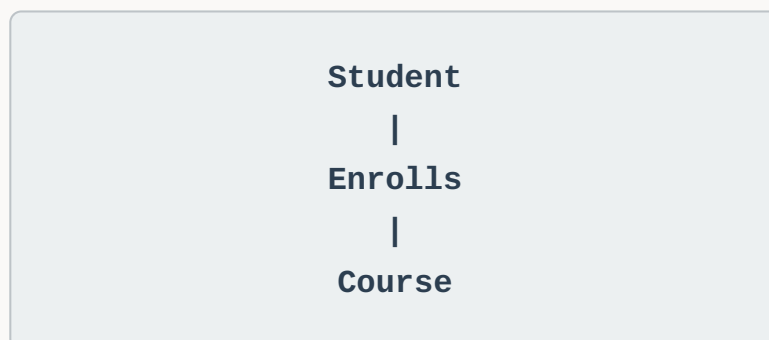
Authored by : Arpit Raj , Lnmiit jaipur

The entire mapping process can be summarized in one table.

ER Concept	Relational Model
Entity	Table
Entity Set	Table
Attribute	Column
Entity Instance	Row
Key Attribute	Primary Key
Relationship	Foreign Key / Junction Table
Composite Attribute	Multiple Columns
Multivalued Attribute	Separate Table
Derived Attribute	Usually Not Stored
Weak Entity	Separate Table with Owner's PK + Partial Key

ER diagrams are excellent for designing databases.

But databases don't store:



A DBMS stores:

- Students Table
- Courses Table
- Enrollments Table

So every conceptual object must be converted into relational tables.
This process is called ER-to-Relational Mapping.

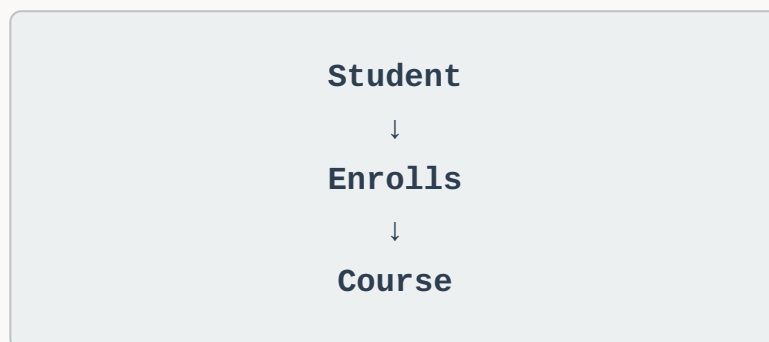
Rule

- Every strong entity becomes one table.
- Every attribute becomes a column.
- Key Attribute → Primary Key , Relationship → Foreign Key , For 1:N relationships, place the Primary Key of the "1" side as a Foreign Key in the "N" side.

Many-to-Many Relationship

This is one of the most important mapping rules.

Example



A student can enroll in many courses.

A course has many students.

Can we place StudentID inside Course?

No.

Can we place CourseID inside Student?

No.

Solution:

Create a new table.

Enrollments

| StudentID | CourseID |

Primary Key

(StudentID, CourseID)

Both are also Foreign Keys.

Every many-to-many relationship becomes a separate junction (bridge/associative) table.

Weak Entity.

Employees

| EmployeeID | Name |

Dependents

| EmployeeID | DependentName | Age |

Primary Key

(EmployeeID, DependentName) , EmployeeID is:

- Foreign Key
- Part of Primary Key , Weak Entity →
Separate table containing:
 - Owner's Primary Key
 - Partial Key
 - Other attributes.

Why Are Multivalued Attributes Placed in Separate Tables?

Because storing multiple values in one column:

- Violates atomicity

- Violates First Normal Form (1NF)
- Makes searching difficult
- Makes indexing inefficient
- Causes update anomalies

A separate table keeps one value per row and preserves relational principles.

Derived attributes are computed from stored attributes. Storing them can lead to redundancy and inconsistencies when the source data changes. That's why we don't store derived attributes